CarlinKit T2C - Tesla Browser Resolution Fix

Custom Firmware Reverse-Engineering, Patching, Rebuilding and Flashing Guide

Tested: CarlinKit T2C - branch `1508` - firmware `26020416.1508.1` - Tina Linux - test viewport 773 x 601.

Device identifiers and the original computer username have been replaced or omitted for safe sharing.

Purpose

This guide documents the successful diagnosis and permanent fix of a CarlinKit T2C scaling/cropping problem that appeared after a Tesla software update. CarPlay loaded, but the UI exceeded the visible Tesla browser area. The built-in HD, SD and Auto settings did not solve it.

The root cause was a `min-width: 1024px` rule in the T2C web UI. The newer Tesla browser exposed a smaller viewport, while the T2C JavaScript transmitted `window.innerWidth` and `window.innerHeight` to its backend during WebSocket initialization.

Important warning

This procedure flashes modified firmware. A bad image, incompatible hardware revision, power interruption or unexpected recovery behavior can brick the adapter. Keep the untouched official firmware and use stable power. These steps were validated only on the firmware branch described here.

0. Prerequisites and tool installation

The workflow below uses macOS and Homebrew. GNU cpio is required because the macOS/BSD cpio may not support the CRC archive format used by this firmware.

```
# Install Homebrew if needed
/bin/bash -c "$(curl -fsSL \
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"

# Required
brew install cpio
brew install squashfs
brew install binwalk

# Optional troubleshooting/discovery
brew install nmap

# Apple Silicon PATH
export PATH="/opt/homebrew/bin:$PATH"
export PATH="/opt/homebrew/opt/cpio/bin:$PATH"

# Verify the actual binaries being used
which -a binwalk
which -a nmap
which -a unsquashfs
which -a mksquashfs

binwalk --version
nmap --version
/opt/homebrew/opt/cpio/bin/cpio --version
unsquashfs -version
mksquashfs -version
```

1. Connect to the T2C

Connect the Mac to the T2C Wi-Fi. The tested adapter was reachable at 192.168.3.1.

```
curl -v http://192.168.3.1/

curl -s \
'http://192.168.3.1/cgi-bin/index.cgi?id=host'
```

The host JSON exposes the firmware version and device-specific values. Replace serial numbers, MAC addresses, IMEI/UUID values and similar identifiers before publishing logs.

2. Check the built-in resolution setting

The frontend maps 1 = HD, 2 = SD and 3 = Auto. All three modes were tested and did not correct the cropping.

```
curl \
'http://192.168.3.1/cgi-bin/index.cgi?id=set&pix=1'
```

3. Download the official firmware

The tested device used firmware branch 1508. Its metadata exposed the official update image.

```
curl -s \
'https://cpbox.oss-cn-shenzhen.aliyuncs.com/1508/version.json'

curl -L \
'https://cpbox.oss-cn-shenzhen.aliyuncs.com/1508/update.img' \
-o ~/Desktop/T2C-update.img

file ~/Desktop/T2C-update.img
ls -lh ~/Desktop/T2C-update.img
xxd -l 256 ~/Desktop/T2C-update.img
```

Do not assume another hardware/firmware branch uses the same URL.

4. Identify the firmware format

Strings and Binwalk reveal an SWUpdate/CPIO package. The CPIO begins at byte 45 (0x2D). The first 45 bytes are an MD5 wrapper.

```
strings -a ~/Desktop/T2C-update.img | head -50
binwalk ~/Desktop/T2C-update.img

# Outer layout:
# <32-char MD5>  update.swu\n
# <CPIO archive at offset 45>
```

5. Extract the firmware

```
binwalk -e ~/Desktop/T2C-update.img

find ~/extractions/T2C-update.img.extracted/2D \
  -maxdepth 2 -type f -ls

# Expected payload:
# sw-description
# customer
# kernel
```

6. Inspect and extract customer

The customer partition is SquashFS 4.0, XZ-compressed, block size 262144 on the tested firmware.

```
file \
~/extractions/T2C-update.img.extracted/2D/customer
```

```
unsquashfs -s \
~/extractions/T2C-update.img.extracted/2D/customer

mkdir -p ~/Desktop/t2c-customer
cd ~/Desktop/t2c-customer

unsquashfs \
~/extractions/T2C-update.img.extracted/2D/customer
```

7. Locate the web interface

The relevant frontend is under squashfs-root/app/www/. Important files include css/styles.css, js/main.js, js/renderer.js and index.in.

8. Identify the actual resolution bug

The original stylesheet forced the body to remain at least 1024 px wide.

```
body {
    font-weight: 400;
    min-width: 1024px;
    font-size: 30px;
}

# main.js also sends:
var h = new Int16Array([
    s.length + v.length + 6,
    2,
    window.innerWidth,
    window.innerHeight
]);
```

9. Reproduce it in Chrome

Open the T2C page, enable Chrome DevTools Device Toolbar and emulate 773 x 601. Run:

```
({
  innerWidth: window.innerWidth,
  innerHeight: window.innerHeight,
  bodyWidth: document.body.getBoundingClientRect().width,
  documentWidth: document.documentElement.getBoundingClientRect().width
})

# Original result:
# innerWidth:    1024
# innerHeight:    797
# bodyWidth:     1024
# documentWidth:  773
```

10. Validate the CSS fix without flashing

```
document.body.style.minWidth = '0px'

# Run the diagnostic again.
# Corrected result:
# innerWidth:    773
# innerHeight:   601
# bodyWidth:     773
# documentWidth: 773
```

This strongly confirmed that the forced 1024 px minimum width was changing the effective viewport.

11. Patch styles.css

Change min-width: 1024px to min-width: 0.

```
body {
```

```
    font-weight: 400;
    min-width: 0;
    font-size: 30px;
}

grep -n 'min-width' \
squashfs-root/app/www/css/styles.css | head
```

12. Patch the resize handler

A secondary bug invoked rf() immediately instead of passing it as the timeout callback.

```
# Original
wst = this.setTimeout(rf(), 1000);

# Modified
wst = this.setTimeout(rf, 1000);

grep -n 'setTimeout(rf' \
squashfs-root/app/www/js/main.js
```

13. Rebuild the customer SquashFS

```
cd ~/Desktop/t2c-customer

mksquashfs squashfs-root customer-mod \
  -comp xz \
  -b 262144 \
  -noappend \
  -xattrs \
  -exports \
  -all-root \
  -nopad

unsquashfs -s customer-mod
stat -f '%z' customer-mod
```

The -nopad option avoids adding default SquashFS block padding that was not present in the original image.

14. Prepare the SWUpdate directory

```
rm -rf ~/Desktop/t2c-custom-ota
mkdir -p ~/Desktop/t2c-custom-ota
cd ~/Desktop/t2c-custom-ota

cp \
~/extractions/T2C-update.img.extracted/2D/sw-description .

cp ~/Desktop/t2c-customer/customer-mod ./customer

cp \
~/extractions/T2C-update.img.extracted/2D/kernel .

chmod 755 sw-description customer kernel
```

15. Rebuild update.swu

Preserve the original order: sw-description, customer, kernel. Use GNU cpio CRC format (070702) and root ownership.

```
printf '%s\n' sw-description customer kernel | \
/opt/homebrew/opt/cpio/bin/cpio \
  -o -H crc -R 0:0 > update-custom.swu

xxd -l 8 update-custom.swu

/opt/homebrew/opt/cpio/bin/cpio \
  -itv < update-custom.swu
```

16. Rebuild the outer update.img

```
MD5=$(md5 -q update-custom.swu)

printf '%s  update.swu\n' "$MD5" \
  > T2C-custom-update.img

cat update-custom.swu >> T2C-custom-update.img

head -c 45 T2C-custom-update.img
echo
binwalk T2C-custom-update.img
```

Binwalk should again locate the CPIO archive at byte 45.

17. Validate before flashing

```
rm -rf ~/Desktop/t2c-final-check
mkdir -p ~/Desktop/t2c-final-check
cd ~/Desktop/t2c-final-check

dd \
  if=~/Desktop/t2c-custom-ota/T2C-custom-update.img \
  of=update-check.swu \
  bs=1 skip=45 2>/dev/null

md5 -q update-check.swu

head -c 32 \
~/Desktop/t2c-custom-ota/T2C-custom-update.img
echo

/opt/homebrew/opt/cpio/bin/cpio \
  -idmv < update-check.swu

shasum -a 256 \
~/Desktop/t2c-customer/customer-mod \
~/Desktop/t2c-final-check/customer
```

The MD5 values must match. The SHA-256 of customer-mod and the customer recovered from the final image must also be identical.

18. Verify the patched files inside the final image

```
mkdir -p customer-check
cd customer-check
unsquashfs ../customer

grep -n 'min-width' \
squashfs-root/app/www/css/styles.css | head

grep -n 'setTimeout(rf' \
squashfs-root/app/www/js/main.js

# Expected:
# min-width: 0;
# wst = this.setTimeout(rf, 1000);
```

19. Observed update architecture

Inspection of sw-description and the updater binary indicated a recovery-based SWUpdate process. The kernel is written to /dev/by-name/boot and customer to /dev/by-name/customer.

```
update.img
  -> validate/extract update.swu
  -> upgrade_recovery
  -> swu_mode=upgrade_kernel
  -> ly_boot_mode=update
```

```
  -> boot_partition=recovery
  -> reboot
  -> write kernel + customer
  -> swu_mode=upgrade_usr
  -> boot_partition=boot
  -> ly_boot_mode=normal
  -> reboot
```

No obvious A/B automatic rollback mechanism was identified during this investigation.

20. Flash the custom firmware

Use stable power. Do not disconnect the T2C after the upload succeeds while the update/reboot process is running.

```
cd ~/Desktop/t2c-custom-ota

curl -v \
  -F 'file=@T2C-custom-update.img;filename=update.img' \
  'http://192.168.3.1/cgi-bin/index.cgi?id=upload'

# Successful response:
# {"code": 0, "msg": "upload successful"}
```

21. Confirm boot after flashing

```
curl -s \
'http://192.168.3.1/cgi-bin/index.cgi?id=host'
```

Receiving the normal device JSON confirms that the web/update subsystem has returned.

22. Validate the permanent fix

Open the T2C page again, emulate 773 x 601 and run the same viewport diagnostic. The post-flash result was:

```
innerWidth:    773
innerHeight:   601
bodyWidth:     773
documentWidth: 773
```

No temporary DevTools CSS override was required, confirming that the modified customer filesystem was active.

23. Final Tesla test

Reconnect the T2C to the Tesla. Confirm that CarPlay fills the available browser area, nothing is clipped, touch coordinates remain aligned, reconnection works, and the fix persists after power cycling.

Technical root-cause summary

```
Tesla viewport: ~773 x 601
        |
Original CSS: body min-width = 1024 px
        |
window.innerWidth becomes 1024
        |
T2C sends innerWidth/innerHeight to backend
        |
CarPlay rendering dimensions can exceed visible area
        |
Interface is cropped

After patch:
body min-width = 0
        |
```

```
innerWidth = 773
innerHeight = 601
        |
browser layout and reported dimensions match
```

Files intentionally modified

```
/mnt/customer/app/www/css/styles.css
/mnt/customer/app/www/js/main.js
```

The original kernel and sw-description were reused. No CarPlay libraries, kernel binaries, bootloader files, certificates or Tesla binaries were intentionally modified.

Privacy checklist before publishing

- Replace serial numbers with <DEVICE_SERIAL>.
- Replace Wi-Fi/Bluetooth MAC addresses with placeholders.
- Remove IMEI, UUID and other device-specific identifiers.
- Remove personal usernames from shell prompts and filesystem paths.
- Review screenshots and terminal output for identifiers before posting.

Tested configuration

Component	Value
Adapter	CarlinKit T2C
Firmware branch	1508
Firmware	26020416.1508.1
Project	ly1508
Platform	Tina Linux
Customer filesystem	SquashFS 4.0 / XZ
Block size	262144
Update archive	SWUpdate / CPIO CRC (070702)
Outer wrapper	MD5 + " update.swu\n"
Test viewport	773 x 601

This is a record of one successful modification path, not a guarantee of compatibility with every CarlinKit T2C hardware or firmware revision.